

Generación de Texto

Módulo 5 · NLP y Modelos de Lenguaje (LLMs)

Temperatura, sampling, beam search, structured outputs y parametros de generacion para cada tipo de caso de uso en produccion.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Generación de Texto con LLMs

La generación de texto es la capacidad que hace a los LLMs más visibles y más útiles: producir texto fluido, coherente y relevante en respuesta a cualquier instrucción. Pero la generación de texto no es un proceso determinista: hay múltiples decisiones probabilísticas en cada paso, y los parámetros que controlan esas decisiones tienen un impacto profundo en la calidad, diversidad y fiabilidad del output. Entender cómo funciona la generación permite diseñar sistemas que producen exactamente el tipo de output que el caso de uso requiere.

En producción, los parámetros de generación son una dimensión crítica de la configuración del sistema, tan importante como el prompt. Un sistema de generación de código necesita temperatura baja y decodificación determinista para producir código correcto y reproducible. Un sistema de brainstorming necesita temperatura alta y variedad para producir ideas diversas. Un sistema de extracción de datos necesita outputs estructurados con forzamiento de esquema para garantizar el parsing correcto.

Los mecanismos de decodificación

Greedy decoding: siempre el token más probable

Greedy decoding selecciona en cada paso el token con mayor probabilidad (argmax de la distribución softmax). Es completamente determinista: el mismo input siempre produce el mismo output. Es rápido y produce texto coherente, pero puede quedar atrapado en repeticiones o en caminos localmente óptimos que no son globalmente mejores. Para extracción de datos estructurados o generación de código donde se necesita máxima consistencia, greedy decoding (equivalente a temperatura 0) es frecuentemente la elección correcta.

Sampling: muestrear de la distribución

En lugar de seleccionar siempre el token más probable, sampling muestrea de la distribución completa. La temperatura T escala los logits antes del softmax: temperatura baja (0.1-0.5) concentra la distribución (menos variedad, más predecible); temperatura alta (0.7-1.2) aplana la distribución (más variedad, más creativo, más propenso a errores). Top-k sampling solo considera los k tokens más probables ($k=50$ es común) antes de muestrear, evitando muestrear tokens muy improbables. Top-p (nucleus) sampling mantiene el conjunto mínimo de tokens cuya probabilidad acumulada supera p ($p=0.9$ es común).

Beam search: explorar múltiples caminos simultáneamente

Beam search mantiene los k caminos más prometedores (beams) en cada paso y expande cada uno para el siguiente token. Al final, devuelve el camino completo con mayor probabilidad conjunta. Con $\text{beam}=5$, el modelo explora 5 caminos en paralelo. Es más costoso computacionalmente que greedy o sampling pero produce texto más globalmente coherente en tareas de traducción y resumen formal. Sin embargo, en la práctica, para los LLMs modernos el sampling con temperatura baja frecuentemente supera a beam search en calidad percibida por

humanos.

SECCIÓN 3 · CÓMO FUNCIONA

Structured outputs: forzar el formato del output

Los LLMs modernos soportan structured outputs (JSON mode): el modelo garantiza que su output es JSON válido conforme a un schema especificado. La API de OpenAI con `response_format={"type": "json_object"}` garantiza JSON válido. Con `response_format` como JSON Schema completo, garantiza además que el JSON tiene la estructura exacta especificada (campos obligatorios, tipos correctos, enumeraciones válidas). Anthropic con Claude permite definir schemas XML y JSON. Esto es fundamental para pipelines de producción donde el output del LLM se parsea programáticamente.

La implementación técnica de structured outputs usa constrained decoding: el proceso de generación se restringe para que solo pueda generar tokens que son compatibles con el schema en el estado actual del JSON (si el campo "tipo" es una enumeración de 3 valores, solo puede generar uno de esos 3 valores). Esto garantiza JSON válido pero puede reducir ligeramente la calidad del contenido si el schema es muy restrictivo.

Repetition penalty y presence/frequency penalty

Los LLMs pueden repetir frases o ideas, especialmente con temperaturas bajas. Presence penalty penaliza todos los tokens que ya han aparecido en el output, reduciendo la repetición general. Frequency penalty penaliza los tokens proporcional a cuántas veces han aparecido, reduciendo más agresivamente los tokens muy repetidos. Para textos largos y conversaciones, estos parámetros previenen la degeneración repetitiva sin sacrificar coherencia.

SECCIÓN 4 · EJEMPLOS REALES

- Extracción de datos de contratos con JSON schema: el LLM recibe el contrato y un schema que especifica {partes: [nombre, rol], fecha_inicio: string, fecha_fin: string, importe: number, moneda: string}. Con structured outputs, el output es garantizadamente JSON parseable con exactamente esos campos, sin necesidad de post-procesamiento para manejar JSON mal formado.
- Generación de correos de marketing con temperatura 0.8: temperatura moderada-alta produce variación natural entre correos para evitar el efecto "correo robótico" que los usuarios detectan. La variedad hace los correos más naturales y con mayor tasa de apertura.
- Sistema de QA sobre documentación técnica con temperatura 0.1: respuestas precisas y consistentes. La misma pregunta produce siempre la misma respuesta, facilitando el testing y la auditoría. La baja temperatura reduce el riesgo de "alucinaciones creativas" en dominios técnicos donde la precisión es crítica.
- Chatbot de soporte con presence_penalty 0.3: reduce la repetición de frases comunes ("Por supuesto, estaré encantado de ayudarte") que hacen el chatbot sonar robótico y degradan la experiencia de usuario.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Usar temperatura 0 para todas las tareas. Temperatura 0 produce el output más probable pero no siempre el más útil. Para tareas generativas donde se necesita variedad o creatividad, temperatura 0 produce outputs repetitivos y menos naturales. Calibra la temperatura según el tipo de tarea.

Error 2: Parsear JSON de un LLM sin usar structured outputs. Si el LLM genera texto con JSON incrustado sin forced structured output, es frecuente recibir JSON mal formado, con campos adicionales, o con el JSON envuelto en markdown (``json...``). Usa siempre el JSON mode o structured output de la API para garantizar parseabilidad.

Error 3: Ignora el max_tokens. Si no se especifica max_tokens, el LLM puede generar respuestas muy largas para tareas simples, añadiendo coste sin valor. Define siempre un max_tokens apropiado para el tipo de tarea y añade instrucciones en el prompt si es necesario para controlar la longitud.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Guía de parámetros de generación para casos de uso comunes: extracción de datos (JSON, campos específicos) → temperatura 0, structured output con JSON schema, max_tokens suficiente para el output. Resumen ejecutivo → temperatura 0.3, top_p 0.9, max_tokens con el límite de palabras deseado. Generación de código → temperatura 0, top_p 1. Chat conversacional → temperatura 0.7, top_p 0.9, presence_penalty 0.3. Brainstorming → temperatura 1.0, top_p 0.95. Clasificación y etiquetado → temperatura 0, logit_bias para restringir el output a las clases válidas.

Para implementar structured outputs en OpenAI API: usa response_format con el schema JSON definido. Con LangChain y otros frameworks: usa with_structured_output() que implementa constrained decoding automáticamente. Con Anthropic: usa el bloque de herramientas con input_schema para forzar el output JSON.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M2-T3 (Probabilidad): la temperatura y el sampling son parámetros de la distribución de probabilidad softmax.
- M6 (Ingeniería de prompts): los parámetros de generación son una dimensión de la configuración del sistema complementaria al diseño del prompt.
- M8 (LLMOps): la latencia, el throughput y el coste de inferencia son directamente afectados por los parámetros de generación (temperatura, max_tokens, streaming).

SECCIÓN 8 · RESUMEN EJECUTIVO

La generación de texto LLM es un proceso de muestreo probabilístico controlado por parámetros. Temperatura (0=determinista, 1=distribución aprendida) controla la creatividad vs consistencia. Top-k y top-p limitan el espacio de muestreo para evitar tokens muy improbables. Structured outputs (JSON mode + schema) garantizan outputs parseables programáticamente: obligatorio en pipelines de producción. Presence/frequency penalty reduce la repetición en textos largos. Guía práctica: T=0 para extracción y código, T=0.3-0.5 para QA precisa, T=0.7-0.9 para chat y texto general, T=1.0+ para creatividad. Define siempre max_tokens. Gestiona el historial en sistemas

conversacionales.